

Language Extensions (Comments, FOR Statements, etc.)

 flexonsbd.blogspot.com/2023/07/for.html

Tuesday, July 4, 2023

Attempts to create a compiler, Part 2: Language extensions (comments, FOR statements, etc.)

Attempts to create a compiler, Part 2: Language extension (Struggling with FOR loops)



(It's included as an icon for display purposes.)

It's been quite a while since the last update, and I'm still struggling to extend the language, but here's the current situation.

The following three features were expanded:

1. The ability to write comments has been enabled.

I believe commenting out is essential when creating programs, so I've made it so that parts enclosed in `/*` and `*/` are treated as comments and skipped over.

2. The format of the PRINT statement (and PRINTLN) has been changed.

Perhaps because I'm used to C, the format of the PRINT statement felt strange to me, so I changed the format to enclose the expression to be printed in parentheses.

Before change: `PRINT I, ":", DATA[I];`

After modification: `PRINT (I, ":", DATA[I]);`

3. Added a FOR statement. (...Incomplete)

While control statements include IF, WHILE, and DO..WHILE, I tried adding a FOR statement because I thought it would be convenient to have one as well.

I managed to implement steps 1 and 2, but adding the FOR loop in step 3 was a real struggle. I finally managed to create something that works somewhat, but it doesn't behave as I expected.

For example, in the case of `FOR (I=0; I<10; I=I+1) PRINT(I,"");`

The result is 1, 2, 3, 4, 5, 6, 7, 8, 9, 10.

The reason for this is that, because we are going through the FOR statement from the beginning, we first set the value of I with the initial setting I=0, then check the truth value of the termination condition I<10, and if it is true, we then increment I=I+1, execute the PRINT statement, and then return to the beginning of the FOR statement. This procedure is repeated, so the control variable is incremented from the very first iteration.

Because this language uses a stack-based approach where everything is pushed onto the stack, it's impossible to save intermediate steps anywhere, and the program is executed sequentially, resulting in this outcome.

(WHILE and DO..WHILE statements are formatted to execute sequentially and produce the correct result.)

In stack languages, is it impossible to have statements like FOR statements that must be executed in an order different from the parsed order? Or is there some technique to achieve this?

(Come to think of it, Professor Nakata's PL/0 doesn't have any FOR statements either...)

Due to my lack of knowledge and experience, it seems I can't go any further, so I'll stop extending the language here. Next, I'll modify the part that converts to an intermediate language to directly convert to 6809 instructions, and then I'll finish it as a compiler that generates code that runs on a 6809 machine.